

FINAL YEAR PROJECT REPORT

Project title:

**Design and development of a multi-agent system for queue
waiting-time estimation from video streams**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	ix
List of Tables	x
General Introduction	xi
1 Context of the Project	1
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 DataDoIt	2
1.2.2 Sectors of Activity	3
1.3 Presentation of the Project	4
1.3.1 Project Context	4
1.3.2 Problem Statement	4
1.3.3 Study of Existing Approaches	4
1.3.4 Proposed Solution	5
1.3.5 Objectives of the Project	7
1.3.6 Project Scope and Boundaries	7
1.3.7 Target Audience	7
1.4 Project Management Methodology	8
1.4.1 Choice of Methodology: Agile	8
1.4.2 SCRUM	9
1.4.2.1 Module A: Backend Core (Sprints 1-3)	9
1.4.2.2 Module B: Web Experience (Sprints 4-5)	9

1.4.2.3	Module C: Automation and Finalization (Sprints 6-8) . .	9
1.5	Conclusion	10
2	Requirements Specification and Analysis	11
2.1	Introduction	12
2.2	Requirements Specification	12
2.2.1	Identification of Actors	12
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	16
2.4	Global Use Case Diagram	16
2.5	Sprint Scope Overview	17
2.6	Data Modeling	18
2.6.1	Core Entities	18
2.6.2	Data Lifecycle and Persistence Strategy	19
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	20
2.8	Software Architecture	22
2.8.1	Overall Architecture	22
2.8.2	Backend Architecture	22
2.8.3	Frontend Architecture	23
2.9	Conclusion	23
3	Sprint 1: Foundation and Perception	24
3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	26

3.3.1	Detailed Use Case Diagram	26
3.3.2	Use Case Scenarios	26
3.4	Design	27
3.4.1	Activity Diagram	27
3.4.2	Sequence Diagrams	27
3.5	Implementation	28
3.5.1	Detection Layer	28
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29
4	Sprint 2: Analytics and Uncertainty	30
4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	32
4.3.3	Sequence Diagrams	33
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	34
4.4.3	Runtime Contract for Monitoring and Automation	34
4.5	Conclusion	34
5	Sprint 3: Advanced Connectivity and API	35
5.1	Introduction	36
5.2	Sprint Backlog	36

5.3	Design	37
5.3.1	Component Diagram	37
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	38
5.4	Implementation	38
5.4.1	Camera Source Integration	38
5.4.2	Camera Discovery and Stream Setup	39
5.4.3	API Layer and Data Contract	39
5.4.4	Live Video Delivery	39
5.4.5	Runtime Control and Live Events	40
5.4.6	Persistence and Recovery	41
5.5	Conclusion	41
6	Sprint 4: Web Design and Scaffolding	42
6.1	Introduction	43
6.2	Sprint Backlog	43
6.3	Design	43
6.3.1	UI and UX Design	43
6.3.2	Frontend Architecture	44
6.3.3	Design Rationale	45
6.4	Implementation	45
6.5	Conclusion	45
7	Sprint 5: Frontend Integration and Live Monitoring	46
7.1	Introduction	47
7.2	Sprint Backlog	47
7.3	Design	48
7.3.1	Data Flow Diagram	48

7.3.2	Sequence Diagrams	48
7.4	Implementation	49
7.4.1	API Integration	49
7.4.2	Dashboard State Orchestration	49
7.4.3	Live Update Reliability	49
7.4.4	Live Video Delivery	50
7.4.5	Dashboard UI Integration	50
7.4.6	Integration Testing	51
7.5	Conclusion	51
8	Sprint 6: n8n Automation and Alerting	52
8.1	Introduction	53
8.2	Sprint Backlog	53
8.3	Design	54
8.3.1	Component Diagram	54
8.3.2	Data Flow Diagram	54
8.4	Implementation	55
8.4.1	Alert Payload Structure	55
8.4.2	Delivery Reliability	55
8.4.3	Alert Dispatch in the Runtime Loop	55
8.4.4	n8n Workflow Routing and Telegram Delivery	56
8.4.5	Workflow Validation	56
8.5	Conclusion	56
9	Sprint 7: Evaluation and Optimization	57
9.1	Introduction	58
9.2	Sprint Backlog	58
9.3	Design	58

9.3.1	Verification Plan	58
9.3.2	Performance Targets	59
9.4	Implementation	59
9.4.1	Verification Coverage	59
9.4.2	Runtime Tuning	59
9.4.3	Fault-Tolerance Improvements	60
9.4.4	Overall Outcome	60
9.5	Conclusion	60
10	General Conclusion	61
10.1	Summary	62
10.2	Objective Review	62
10.3	Future Work	63
A	Appendices	65
A.1	Useful Commands	65
A.2	Example Webhook Payload	65
A.3	Items to Add Before Final Submission	66



List of Figures

2.1	Global use case diagram of the queue wait-time estimation system. . . .	17
2.2	Unified SQLite schema used for feed metadata, queue sessions, and alert archival.	20
3.1	Sprint 1 use case diagram for detection and tracking in queue zones. . .	26
3.2	Sprint 1 activity flow from frame ingestion to queue-zone updates. . . .	27
3.3	Sprint 1 sequence diagram from video ingestion to queue-state updates.	27
4.1	Sprint 2 components for queue analysis and stability handling.	31
4.2	Data flow from tracked events to queue metrics and stability state. . . .	32
4.3	Sprint 2 sequence diagram for queue rate estimation and stability handling.	33
5.1	Sprint 3 components for camera integration and backend services.	37
5.2	Data flow from video feeds to API responses and WebSocket broadcasts.	37
5.3	Sprint 3 sequence diagram for feed orchestration and live event distribution.	38
6.1	Frontend architecture for routing, components, and state boundaries. . .	44
7.1	Data flow between backend services and frontend monitoring components.	48
7.2	Sprint 5 sequence diagram for dashboard updates and alert surfacing. .	48
8.1	Components involved in backend-to-n8n-to-Telegram automation flow.	54
8.2	Alert payload flow from backend metrics to external notifications. . . .	54



List of Tables

1.1	Collaboration perimeter with DataDoIt	3
1.2	From operational pain points to system-level requirements	5
1.3	Positioning of the proposed solution against common approach families	6
1.4	Objective-to-module traceability	8
1.5	Scrum module mapping used in this report	10
2.1	Actor responsibility matrix	13
2.2	Functional requirement traceability matrix	14
2.3	Non-functional requirement quality matrix	15
2.4	Backlog domain structuring	16
2.5	Primary global use cases	17
2.6	Project sprint scope overview	18
2.7	Conceptual entity mapping	19
2.8	Hardware environment	20
2.9	Execution environment profile	21
2.10	Architecture layer-to-module mapping	23



General Introduction

Service organizations are under constant pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers all face the same problem: queues change quickly, are difficult to monitor by hand, and react strongly to demand fluctuations.

In practice, customers often join a queue without knowing how long they will wait, while managers and operational staff rarely have reliable real-time indicators for adjusting resources quickly. Manual observation is subjective, and simple counting tools are not enough to estimate waiting time in a meaningful way.

This end-of-studies project addresses that gap through the design and development of a real-time queue monitoring system based on computer vision and queueing theory. The proposed system detects and tracks people in video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes metrics that can be used operationally.

It combines classical queueing theory with computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

This report is organized as follows. Chapter 1 presents the project context, host organization, problem statement, and objectives. Chapter 2 defines the requirements analysis and the main architecture choices. Chapters 3 to 9 follow the implementation sprint by sprint, from the perception foundations to evaluation and optimization. Chapter 10 concludes the report and presents future work directions.

Context of the Project

Table of Content

1.1	Introduction	2
1.2	Host Organization	2
1.2.1	DataDoIt	2
1.2.2	Sectors of Activity	3
1.3	Presentation of the Project	4
1.3.1	Project Context	4
1.3.2	Problem Statement	4
1.3.3	Study of Existing Approaches	4
1.3.4	Proposed Solution	5
1.3.5	Objectives of the Project	7
1.3.6	Project Scope and Boundaries	7
1.3.7	Target Audience	7
1.4	Project Management Methodology	8
1.4.1	Choice of Methodology: Agile	8
1.4.2	SCRUM	9
1.5	Conclusion	10

1.1 Introduction

This chapter places the Queue Wait-Time Estimation System in its operational, organizational, and methodological context. After a short introduction, it explains why queue monitoring remains an important problem for service organizations and why an end-to-end technical solution is needed rather than isolated counting tools.

The chapter also defines the baseline used throughout the report: the collaboration context with DataDoIt, the problem formulation, the comparison with existing approaches, the solution scope, and the Scrum-based organization of the work. This level of detail matters because the later technical chapters rely on these assumptions.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt works at the intersection of data science and sustainability, helping governments, municipalities, and enterprises turn environmental commitments into measurable results across the EMEA region. The company offers services that cover decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. Its motto, Measure It, Optimize It, Do It, reflects a practical methodology based on analysis, planning, and technical deployment. To date, the company has delivered more than 15 active projects across 6 sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration provided a concrete setting for work centered on data collection, real-time monitoring, and automated analytics, and these concerns directly shaped the design and scope of the project.

Table 1.1: Collaboration perimeter with DataDoIt

Dimension	Scope in this project
Business context	Queue management in service scenarios where waiting time and staffing decisions are operationally sensitive.
Engineering constraints	Multi-source video ingestion, variable visual conditions, low-latency monitoring, and integration with automation channels.
Expected outputs	Actionable queue indicators, wait-time estimation, and alert-ready payloads for external workflows.
Validation mode	Iterative sprint reviews using implementation evidence from backend modules, API contracts, frontend behavior, and automation tests.

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,
- financial services firms navigating ESG compliance and green finance opportunities,
- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is often subjective, hard to scale, and dependent on continuous attention. In many environments, teams react only after congestion appears instead of anticipating it early with measurable indicators.

This project addresses that gap by turning camera streams into decision-oriented telemetry. The target output is not limited to occupancy counts, but includes people in zone, arrival rate, service rate, expected wait time, queue stability state, and alert states that can be consumed by monitoring and automation layers.

1.3.2 Problem Statement

Existing operational practices and basic counting tools present recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a consequence, interventions (staff reallocation, lane opening, escalation) can be delayed or triggered on incomplete signals. The problem is therefore both analytical and integration-related: it is not enough to produce robust indicators; they also need to be delivered through actionable channels.

1.3.3 Study of Existing Approaches

Recent computer-vision frameworks provide strong building blocks for person perception, notably YOLO-based detection and tracking algorithms such as ByteTrack [2],

Table 1.2: From operational pain points to system-level requirements

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Stability state is not explicit during bursty periods.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Stable/unstable queue-state classification with fallback wait-time logic.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

[3]. Queueing theory, in parallel, offers formal models for service-system behavior, including interpretable relationships between arrival rate, service rate, and expected waiting time [1].

In practice, many deployments remain incomplete. Some focus only on counting and ignore identity continuity; others provide visualization without defining stability clearly; and others stop at analytics without connecting to alert delivery. The result is often a set of disconnected tools instead of one coherent architecture.

1.3.4 Proposed Solution

The proposed system combines perception, analytics, and integration in one modular pipeline:

1. person detection and multi-object tracking from camera feeds,
2. polygon-based queue-zone filtering,

Table 1.3: Positioning of the proposed solution against common approach families

Approach family	Strengths	Typical limitation
Manual or occupancy-only monitoring	Simple to deploy and easy to interpret at first glance.	Weak predictive capability and high human subjectivity.
Vision detection without persistent tracking	Fast frame-level counting.	Identity discontinuity can distort arrival and departure logic.
Analytical queue models without vision coupling	Strong mathematical interpretability.	Requires reliable event extraction pipeline from real scenes.
Proposed integrated pipeline	Couples perception, queue analytics, web monitoring, and automation.	Requires careful multi-layer integration and validation.

3. estimation of λ , μ , and expected wait time,
4. stable/unstable queue-state interpretation for safe operational decisions,
5. real-time web supervision through tiered live transport (WebRTC primary, MJPEG fallback),
6. automation through n8n webhooks and Telegram alerts.

This architecture aims to convert visual observations into operational actions while keeping each layer independently testable. Detection and tracking modules can evolve without rewriting queue formulas, and the web and automation channels can evolve

without changing the core metric contract.

1.3.5 Objectives of the Project

The project objectives are to:

1. detect and track people in real time,
2. count people in configurable queue zones,
3. estimate arrival and service rates,
4. compute expected wait time,
5. classify queue stability for operational reliability,
6. expose metrics through web interfaces and automation workflows.

To keep the implementation traceable, the objectives are mapped to delivery modules and observable outputs.

1.3.6 Project Scope and Boundaries

The implementation scope covers backend perception and analytics modules, API/runtime integration, web monitoring, and external workflow automation. The report deliberately focuses on this web-first operational architecture.

All implementation details and validation narratives are presented through the backend, web, and automation delivery tracks to maintain a consistent production-oriented scope.

1.3.7 Target Audience

The system targets three stakeholder groups with complementary goals:

- **Administrators:** manage manager accounts and access governance.

Table 1.4: Objective-to-module traceability

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Queue stability handling	Module A	Stable/unstable state classification and robust fallback wait-time behavior.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

- **Managers:** configure operational settings, monitor live queue status, react to alerts, and use indicators for staffing and service-quality planning.
- **End users:** benefit indirectly from reduced waiting times and smoother service flow.

This separation is important because each group consumes different levels of abstraction, from per-feed live events to aggregated performance trends.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected to deliver functional increments quickly while reducing integration risks early. The project combines multiple technical domains (computer

vision, queue analytics, backend APIs, frontend workflows, and automation), each with uncertainties that are difficult to resolve using a single monolithic delivery cycle.

Iterative delivery made it possible to validate contracts progressively: first perception outputs, then analytical reliability, then API/runtime integration, then web usability, and finally workflow automation and hardening.

1.4.2 SCRUM

Development followed sprint-based Scrum planning with iterative validation. For report readability and implementation traceability, work is grouped into three logical modules.

1.4.2.1 Module A: Backend Core (Sprints 1-3)

This module established the vision and analytics backbone:

- Sprint 1: detection, tracking, and zone filtering foundation,
- Sprint 2: queue metrics and stability handling,
- Sprint 3: camera connectivity and backend API exposure.

1.4.2.2 Module B: Web Experience (Sprints 4-5)

This module focused on the web supervision experience:

- Sprint 4: web design system and frontend scaffolding,
- Sprint 5: frontend integration with live backend metrics.

1.4.2.3 Module C: Automation and Finalization (Sprints 6-8)

This module covered integration hardening and closure:

- Sprint 6: n8n automation and Telegram alerting,
- Sprint 7: evaluation and optimization,

- Sprint 8: report finalization and project consolidation.

Table 1.5: Scrum module mapping used in this report

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 3	Vision pipeline, queue analytics, camera connectivity, API services	Technical correctness of perception and metrics contracts.
Module B: Web Experience	4 to 5	Frontend design system, dashboard scaffolding, live integration	Usability, interaction continuity, and live data rendering stability.
Module C: Automation and Finalization	6 to 8	n8n alerting, optimization, validation, and reporting	End-to-end operability, alert reliability, and delivery readiness.

1.5 Conclusion

This chapter set the project context, formalized the addressed operational problem, and justified the selected delivery strategy. Chapter 2 builds on this baseline by defining the requirements, architecture constraints, and analysis artifacts used to guide implementation decisions.

Table of Content

2.1	Introduction	12
2.2	Requirements Specification	12
2.2.1	Identification of Actors	12
2.2.2	Functional Requirements	13
2.2.3	Non-functional Requirements	14
2.3	Product Backlog	16
2.4	Global Use Case Diagram	16
2.5	Sprint Scope Overview	17
2.6	Data Modeling	18
2.6.1	Core Entities	18
2.6.2	Data Lifecycle and Persistence Strategy	19
2.7	Project Environment	20
2.7.1	Hardware and Software Environment	20
2.7.2	Technologies Used	20
2.8	Software Architecture	22
2.8.1	Overall Architecture	22
2.8.2	Backend Architecture	22
2.8.3	Frontend Architecture	23
2.9	Conclusion	23

2.1 Introduction

This chapter turns the project objective into the requirements that guided the implementation. We identify the actors, list the functions and quality constraints they need, and show how those requirements are spread across the implementation plan.

Rather than treating the requirements as an isolated checklist, we connect them to the queue-monitoring pipeline used throughout the project: video ingestion, person detection and tracking, queue-metric estimation, dashboard delivery, and alert automation. That link matters because the later chapters follow the same path from requirement to implementation.

2.2 Requirements Specification

2.2.1 Identification of Actors

The system has two actors whose responsibilities reflect the two main operating levels of the platform: governance and supervision. Both roles use the same application, but they do so with different permissions and goals.

The actors of the system are:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.

This split makes the functional scope easier to explain and validate, because access-control flows and operational monitoring flows can be tested separately.

Table 2.1: Actor responsibility matrix

Actor	Main responsibilities	Typical interactions
Administrator	Create, update, and delete manager accounts; enforce access governance rules.	Uses account-management and audit-oriented flows to maintain authorized manager access.
Manager	Configure feeds, zones, and thresholds; monitor live queue status; react to warning/critical alerts; review trends and performance summaries.	Uses setup workflows, dashboard KPIs, live feed views, alert panels, and analytics views for operational decision-making.

2.2.2 Functional Requirements

At a high level, the system captures a feed, counts people, estimates queue behavior, updates the dashboard, and sends alerts when needed. From that workflow, the system must provide the following capabilities:

- real-time person detection and tracking,
- configurable queue zone definition,
- estimation of arrival rate (λ), service rate (μ), and wait time (W),
- queue stability classification with robust wait-time fallback behavior,
- real-time web dashboard updates and alerts,
- RTSP and ONVIF camera integration,
- automated webhook and Telegram notifications,
- historical data logging and analytics.

These requirements define the minimum end-to-end behavior expected from the platform. To keep the implementation trackable, we formalize them in a traceability matrix.

Table 2.2: Functional requirement traceability matrix

ID	Requirement statement	Priority	Verification strategy
FR-01	Detect people from live or recorded video streams in near real time.	High	Validate detector outputs and runtime overlays in the perception layer.
FR-02	Maintain identity continuity through tracking for event-level queue analysis.	High	Validate tracker ID continuity in representative runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion behavior in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in the analytics layer.
FR-05	Classify stable/unstable queue states and apply safe fallback wait-time logic.	High	Validate stability branches and fallback behavior in queue analytics.
FR-06	Expose feed control and live updates through API and websocket channels.	High	Validate endpoint and runtime behavior through API and integration tests.
FR-07	Dispatch alert payloads to external automation workflows.	Medium	Validate webhook dispatch and n8n branch behavior using script-based scenarios.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate SQLite persistence behavior through runtime execution and database checks.

2.2.3 Non-functional Requirements

In addition to functional behavior, the platform must satisfy quality constraints that keep it usable in a live monitoring environment. Because the project combines computer vision, analytics, a web dashboard, and automation workflows, performance and reliability are as important as correctness. For this project, the main quality constraints are:

- **Performance:** maintain real-time behavior and low alert latency.
- **Scalability:** support multiple feeds and independent feed lifecycle control.

- **Reliability:** tolerate transient source failures and integration interruptions.
- **Usability:** provide clear web workflows for monitoring and configuration.
- **Security:** protect integration channels and service endpoints.
- **Maintainability:** keep modular code organization and testable interfaces.

We treat these quality attributes as engineering contracts rather than only design intentions.

Table 2.3: Non-functional requirement quality matrix

ID	Quality attribute	Target indicator	Validation approach
NFR-01	Performance	Stable near-real-time loop and bounded alert propagation delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Independent lifecycle control for multiple feed contexts.	Feed registry behavior and state-transition verification.
NFR-03	Reliability	Graceful handling of camera drops and webhook transmission failures.	Reconnect paths and retry/fallback test scenarios.
NFR-04	Usability	Clear setup-to-monitoring flow for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Controlled integration access via shared secret and credential isolation.	Webhook secret validation and credentialized channel checks.
NFR-06	Maintainability	Modular architecture with explicit contracts and targeted tests.	Code organization review and test-suite coverage by module.

2.3 Product Backlog

The product backlog was prioritized according to business value and technical dependency. To keep the implementation easy to follow, we grouped the items into three domains that mirror the system architecture: backend processing, web supervision, and automation.

- **Backend domain:** perception pipeline, queue analytics, API and stream interfaces.
- **Web domain:** dashboard UX, live monitoring integration, zone editing workflows.
- **Automation domain:** n8n workflow integration, alert routing, webhook reliability.

The first sprints focused on backend foundations, because the dashboard and automation layers depend on a stable metric contract. This order reduced integration risk and made later validation clearer.

Table 2.4: Backlog domain structuring

Domain	Representative backlog scope	Dependency rationale
Backend	Detection/tracking, zone logic, queue formulas, and feed runtime services.	Provides source-of-truth metrics consumed by all upper layers.
Web	Dashboard pages, typed API integration, websocket synchronization, feed controls.	Depends on stable backend contracts and event schemas.
Automation	Webhook payload contract, routing, cooldown, external notifications, archival.	Depends on reliable alert semantics and validated thresholds.

2.4 Global Use Case Diagram

The global use case diagram gives a high-level view of how the administrator and manager interact with the platform during configuration, monitoring, and alert handling.

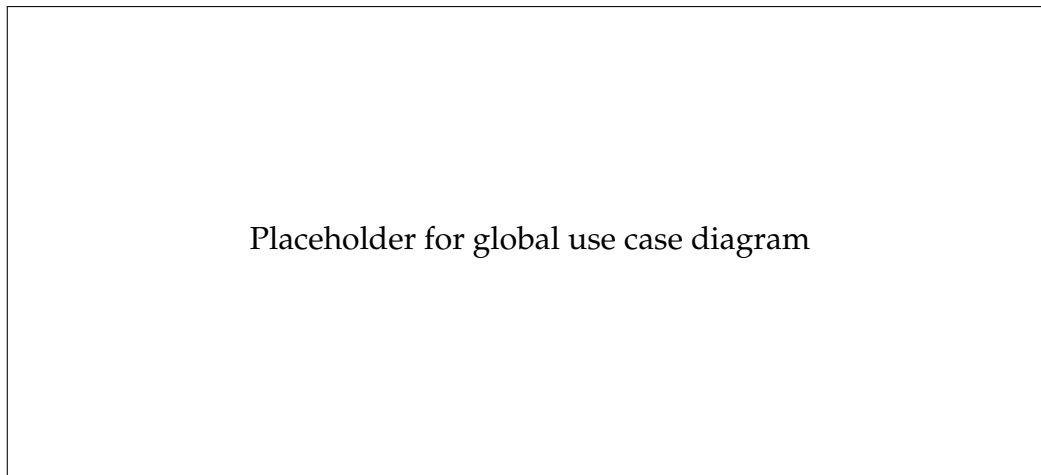


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At the system level, the use cases can be grouped into four interactions: configuration, live monitoring, analytics interpretation, and alert handling. The table below lists the use cases that should appear in the diagram and the main actor responsible for each one.

Table 2.5: Primary global use cases

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Review trends and performance indicators.	Manager

2.5 Sprint Scope Overview

We organized the project into eight sprints so that the pipeline could be delivered step by step and checked at each stage. Table 2.6 summarizes the main outcome of each sprint and the lens used to evaluate it.

Table 2.6: Project sprint scope overview

Sprint	Main outcome	Representative validation lens
Sprint 1	Detection, tracking, and queue-zone perception foundation	Perception correctness and tracker continuity
Sprint 2	Queue analytics and stability handling	Formula validity and stability-branch consistency
Sprint 3	Camera connectivity and backend API exposure	Runtime control and contract stability
Sprint 4	Web design system and frontend scaffolding	UX structure and component baseline
Sprint 5	Frontend integration and live monitoring	API/websocket synchronization quality
Sprint 6	n8n automation and Telegram alerting	Alert routing and delivery reliability
Sprint 7	Evaluation and optimization	Test depth and runtime hardening
Sprint 8	Final consolidation, reporting, and delivery	End-to-end consistency and documentation quality

2.6 Data Modeling

The data model combines runtime monitoring entities with persistence-oriented entities. Its role is to keep live telemetry, configuration data, and alert history aligned so that the system can support both immediate supervision and later analysis without the modules drifting apart.

2.6.1 Core Entities

The main objects are:

- **VideoFeed**: source identity, connectivity, runtime status, and configuration.
- **FeedTransportCapabilities**: per-feed readiness flags for WebRTC and MJPEG routing.
- **QueueMetrics**: timestamped indicators (people in zone, λ , μ , W , stability).

- **Alert:** severity, threshold context, and notification payload.
- **Establishment and Cashier:** business metadata used to organize feeds.

These entities mirror the runtime contract used by the backend so that live metrics, configuration data, and persisted records stay consistent across modules.

Table 2.7: Conceptual entity mapping

Entity	Core semantics	Producer/consumer context
VideoFeed	Declares source type, identity, status, and runtime control metadata.	Produced by onboarding/configuration flows; consumed by runtime and dashboard layers.
FeedTransportCapabilities	Declares transport readiness (WebRTC and MJPEG), source mode, and unavailability reason for each feed.	Produced by API runtime transport endpoint; consumed by dashboard playback routing and fallback logic.
QueueMetrics	Captures analytical outputs used for monitoring and decision support.	Produced by analysis loop; consumed by websocket/API/dashboard and alert logic.
Alert	Encodes threshold breach context and notification-ready payload content.	Produced by threshold evaluation; consumed by webhook automation and archival paths.
Establishment / Cashier	Binds feeds to business-level organization context.	Produced by metadata management APIs; consumed by setup and filtering interfaces.

2.6.2 Data Lifecycle and Persistence Strategy

Runtime metrics are generated continuously and distributed through live channels, while selected records are persisted for auditability and trend analysis. This dual-path strategy improves resilience because temporary integration failures do not imply complete data loss.

SQLite persistence stores operational metrics, structured metadata, and alert archives in a single source of truth. That supports operational continuity as well as post-run diagnosis.

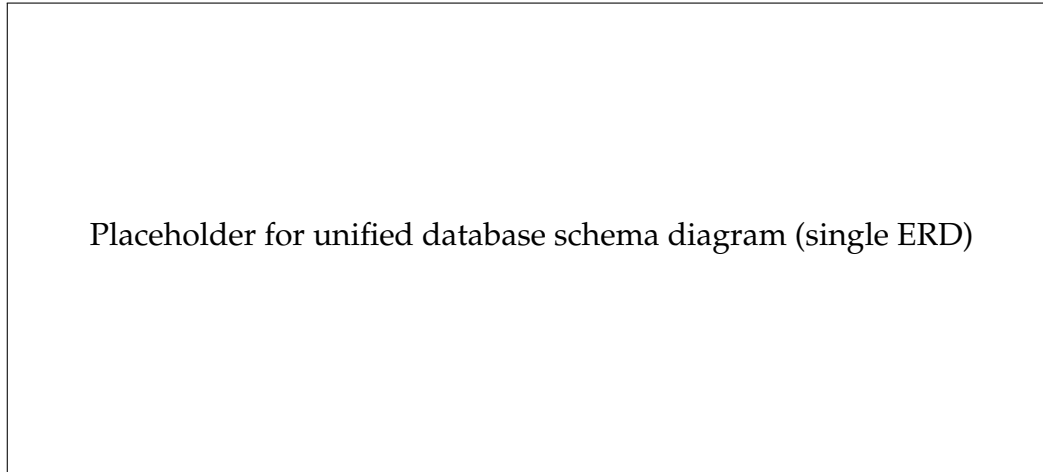


Figure 2.2: Unified SQLite schema used for feed metadata, queue sessions, and alert archival.

2.7 Project Environment

2.7.1 Hardware and Software Environment

The hardware environment used during development is shown in Table 2.8. The software and runtime layers are summarized in Table 2.9.

Table 2.8: Hardware environment

Component	Type
Processor	Intel Core i5 12th Generation
Memory (RAM)	16 GB
Graphics card	NVIDIA GeForce RTX 4060
Storage	512 GB SSD

2.7.2 Technologies Used

The technology stack follows the main stages of the system:

Table 2.9: Execution environment profile

Environment layer	Main tooling	Purpose
Backend runtime	Python ecosystem with vision and scientific libraries	Perception, queue analytics, and integration dispatch.
Frontend runtime	Node/Bun-based TypeScript toolchain with Vite	Dashboard rendering, interaction workflows, and live-state synchronization.
Automation runtime	Self-hosted n8n with webhook endpoints	Alert routing, cooldown logic, and external notifications.
Persistence/runtime support	SQLite database store (metrics, metadata, alerts)	Historical analysis and system state continuity.

- **Vision pipeline:** YOLO26 (object detection model), ByteTrack (multi-object tracker), OpenCV (image processing and camera utilities) [2], [3], [4].
- **Queue analytics:** NumPy (numerical arrays), Pandas (table-oriented data), SciPy (statistical routines) and M/M/1 modeling (basic queueing theory) [1].
- **Backend API:** FastAPI (chosen for its async support and automatic OpenAPI documentation generation) with Pydantic v2 (typed data validation and parsing).
- **Live transport:** WebRTC signaling via MediaMTX WHEP gateway (low-latency streaming) with MJPEG fallback (simple HTTP frame stream) [5], [6].
- **Frontend:** React (component UI library), TypeScript (typed JavaScript), Vite (fast dev/build tool), Tailwind CSS (selected for rapid UI development with a consistent design system).
- **Automation:** n8n workflows (visual automation engine) with Telegram notifications (messaging alerts).
- **Persistence:** SQLite (lightweight relational database for metrics, metadata, and alert archival).

Each component was selected because it fits one layer of the system and keeps the implementation modular from perception through automation.

2.8 Software Architecture

2.8.1 Overall Architecture

We describe the architecture as four stages that match the flow of data through the system:

1. **Video acquisition:** local files, RTSP streams, and ONVIF discovery.
2. **Perception:** detection, tracking, and queue-zone filtering.
3. **Analysis:** queue metrics estimation and stability handling.
4. **Output and integration:** web dashboard, webhook dispatch, and alert automation.

This separation keeps perception, analysis, the user interface, and automation independently testable. For example, queue formulas depend on perception events, but they do not depend on the frontend or notification delivery.

2.8.2 Backend Architecture

The backend exposes REST and websocket interfaces for feed lifecycle control and live metrics. It includes modules for detection orchestration, tracking, queue analysis, threshold evaluation, webhook delivery, and camera-source management.

Feed-level orchestration lets each camera be started, stopped, or refreshed independently. If one feed fails, the others keep running, and the dashboard still receives a predictable event stream.

We treat persistence and integration as core responsibilities rather than add-ons: SQLite-backed state preserves operational context, while webhook delivery turns queue events into external notifications.

Table 2.10: Architecture layer-to-module mapping

Layer	Responsibilities	Representative components
Video acquisition	Source opening, stream transport handling, reconnect behavior.	Source-access and camera-adapter layers
Perception	Person detection, tracking continuity, zone-aware filtering.	Detection, tracking, and zone-management layers
Analysis	Event-window rates, wait-time logic, and stability-aware fallback handling.	Queue analysis and stability-handling logic
Output/integration	API/websocket exposure, webhook dispatch, persistence, dashboard consumption.	API runtime, webhook delivery, and frontend integration layers

2.8.3 Frontend Architecture

The frontend is built as a component-based interface with typed API integration, feed-oriented dashboard composition, and websocket-driven synchronization. Data access is isolated in hooks and client utilities, while pages and reusable components focus on presentation and interaction.

That separation keeps API changes localized to the integration layer and lets the visual components stay stable even when the backend contract changes. It also makes the dashboard easier to extend when new feed controls or indicators are added.

2.9 Conclusion

This chapter establishes the functional baseline, the quality constraints, and the architecture choices that guide the rest of the report. The next chapters turn these requirements into perception, connectivity, automation, and evaluation work.

Sprint 1: Foundation and Perception

Table of Content

3.1	Introduction	25
3.2	Sprint Backlog	25
3.3	Sprint Analysis	26
3.3.1	Detailed Use Case Diagram	26
3.3.2	Use Case Scenarios	26
3.4	Design	27
3.4.1	Activity Diagram	27
3.4.2	Sequence Diagrams	27
3.5	Implementation	28
3.5.1	Detection Layer	28
3.5.2	Tracking Layer	28
3.5.3	Zone-Aware Filtering	28
3.5.4	Main Loop Integration	28
3.6	Conclusion	29

3.1 Introduction

Sprint 1 built the first perception flow of the system. It takes a video stream, identifies people, keeps track of them across frames, and limits counting to the queue area. The goal of this sprint was to prepare clean and reliable input for the later analysis stages.

3.2 Sprint Backlog

The Sprint 1 backlog focused on the basic perception flow:

- Detect people in the video stream while keeping the detector adjustable to different machines.
- Keep the same person identifiable across frames so the counts stay consistent.
- Define queue zones as polygons and count only the people inside these zones.
- Connect capture, detection, tracking, zone filtering, and display into one continuous flow.
- Allow basic runtime settings so the pipeline can be adapted without changing the code.

3.3 Sprint Analysis

3.3.1 Detailed Use Case Diagram

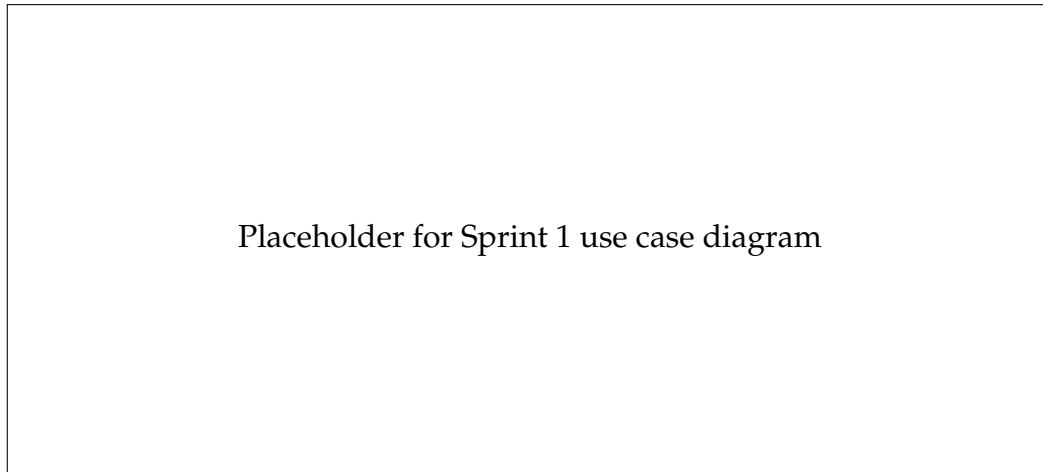


Figure 3.1: Sprint 1 use case diagram for detection and tracking in queue zones.

The use case starts when a manager sets up a source and defines the queue area. After that, the runtime processes each frame and updates the queue state. This gives the later analytics layers a stable perception base to work with.

3.3.2 Use Case Scenarios

Three key scenarios are implemented in Sprint 1:

1. **Person entry into queue zone:** when a person enters the queue area, the system treats that person as a possible arrival.
2. **Consistency across frames:** when the same person appears in consecutive frames, the system keeps the identification stable and avoids double counting.
3. **Zone filtering under mixed scenes:** people outside the queue area are ignored so the queue state stays focused on the relevant part of the scene.

3.4 Design

3.4.1 Activity Diagram

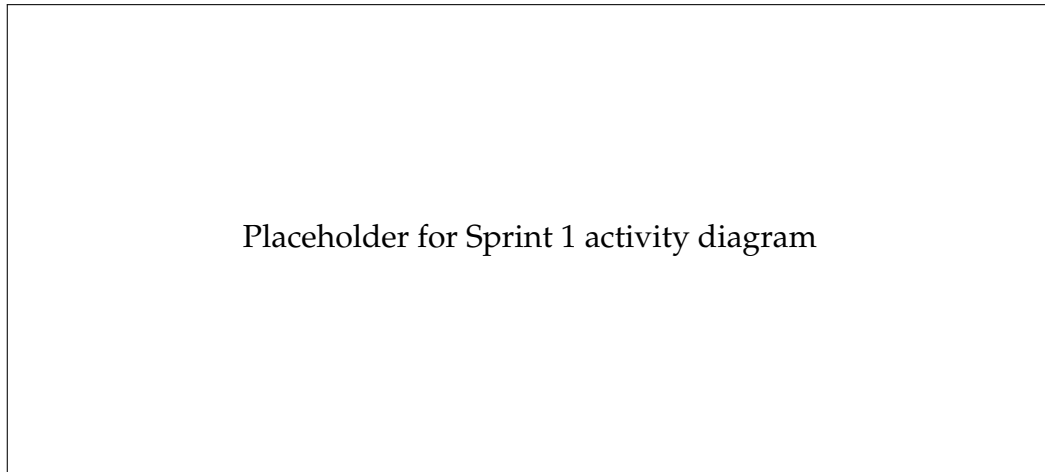


Figure 3.2: Sprint 1 activity flow from frame ingestion to queue-zone updates.

The activity starts from a video source, whether it is a local file or a live camera stream. The system then detects people, keeps track of them, checks whether they are inside the queue area, and passes the filtered result to the next analysis stage.

3.4.2 Sequence Diagrams

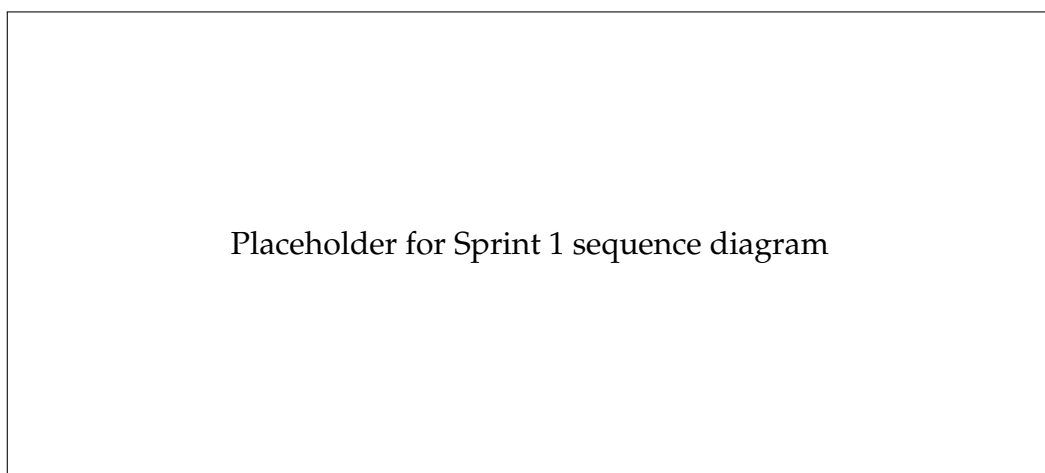


Figure 3.3: Sprint 1 sequence diagram from video ingestion to queue-state updates.

In practice, each frame moves through source handling, detection, tracking, zone checking, and then the queue analysis stage. That ordered flow is what lets Sprint 1 feed the later metric work.

3.5 Implementation

3.5.1 Detection Layer

The detection layer focuses on finding people in each frame while keeping the process adjustable for different machines. It is tuned to balance speed and accuracy, so the system stays responsive without losing the main detections that matter for the queue.

3.5.2 Tracking Layer

The tracking layer keeps the same person identifiable from one frame to the next. This is what makes arrival and departure counts more reliable than simply reading detections in a single frame [3].

3.5.3 Zone-Aware Filtering

Zone logic defines the queue area as a polygon on the frame and keeps only the people inside it. This avoids counting unrelated movement elsewhere in the scene and makes the measurement more relevant to the waiting line.

3.5.4 Main Loop Integration

The pipeline integration is handled in the runtime orchestration layer. The runtime supports:

- handling local files, webcams, and live camera streams,
- adjusting how often frames are processed so the system stays responsive,

- showing detections, IDs, the queue zone, and metrics on screen,
- running without a visible interface when needed for later integration.

The sprint left behind a stable perception baseline that later sprints could build on for queue analytics and automation integration [4].

3.6 Conclusion

Sprint 1 delivered the first working perception layer: people can be detected, followed across frames, and counted only inside the queue area. That gives Sprint 2 cleaner input for rate estimation and wait-time analysis.

Table of Content

4.1	Introduction	31
4.2	Sprint Backlog	31
4.3	Design	31
4.3.1	Component Diagram	31
4.3.2	Data Flow Diagram	32
4.3.3	Sequence Diagrams	33
4.4	Implementation	33
4.4.1	Queue Event Windows and Rate Estimation	33
4.4.2	Wait-Time Logic and Stability Handling	34
4.4.3	Runtime Contract for Monitoring and Automation	34
4.5	Conclusion	34

4.1 Introduction

Sprint 2 moved the project from simple counting to queue interpretation. At this stage, the system starts estimating how people arrive, how quickly service is handled, and how long waiting should take. It also decides when the queue is no longer behaving in a stable way.

4.2 Sprint Backlog

The Sprint 2 backlog focused on turning tracking results into usable queue indicators:

- estimate queue rates from people tracked inside the queue area,
- compute wait time while handling stable and unstable cases,
- protect the calculations from empty windows and division-by-zero problems,
- provide a clear metric output that later layers can reuse.

4.3 Design

4.3.1 Component Diagram

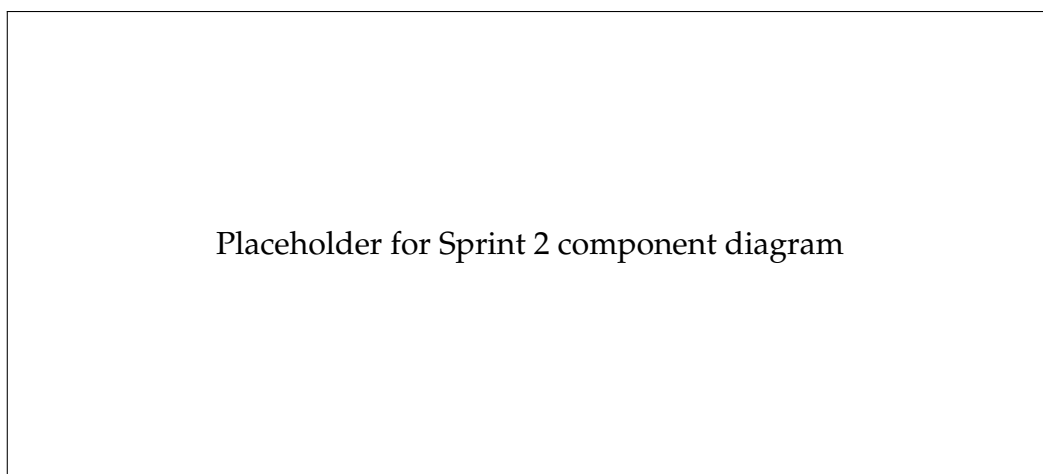


Figure 4.1: Sprint 2 components for queue analysis and stability handling.

At the component level, the analysis module receives the people already identified inside the queue zone and turns them into queue indicators that can be reused by logs, dashboard updates, and alert messages.

4.3.2 Data Flow Diagram

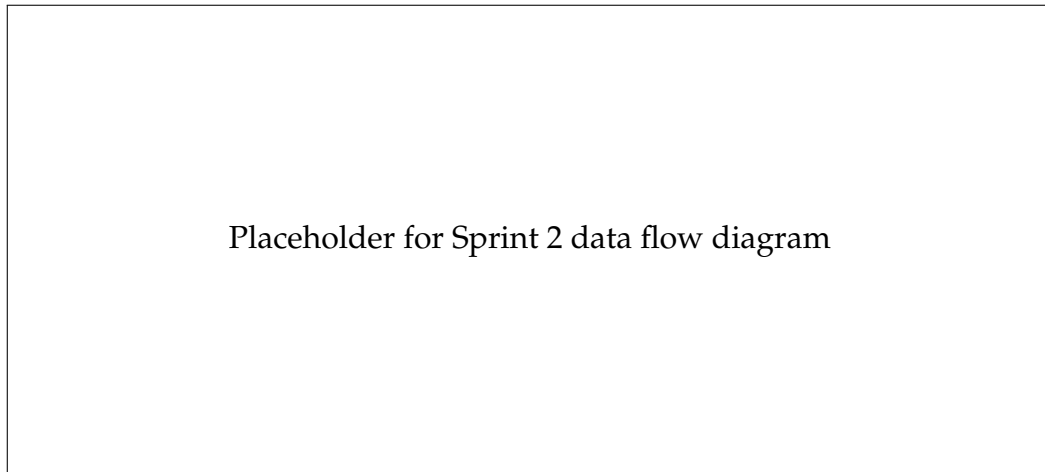


Figure 4.2: Data flow from tracked events to queue metrics and stability state.

Data flow starts from changes in tracked people. Those changes are grouped over short windows to estimate arrival and service rates. The rates are then combined with the current occupancy to produce wait time and a stability label.

4.3.3 Sequence Diagrams

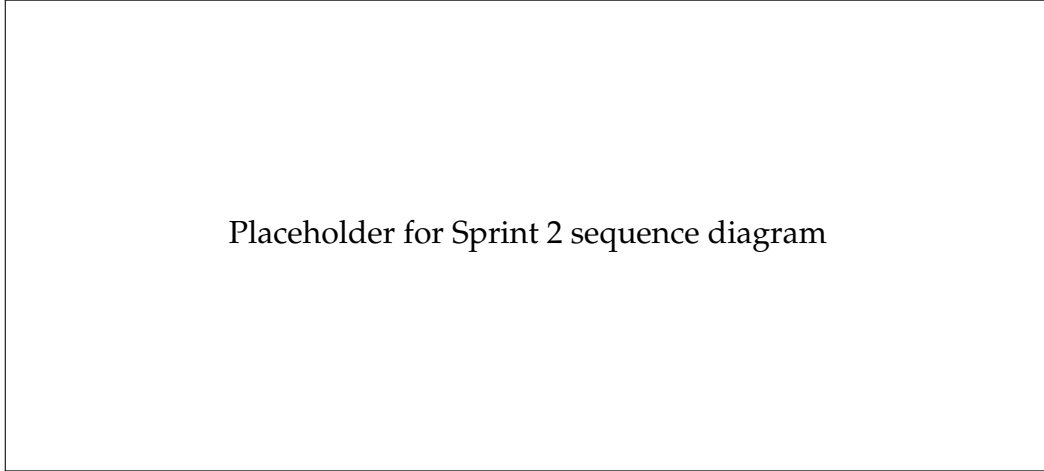


Figure 4.3: Sprint 2 sequence diagram for queue rate estimation and stability handling.

In practice, the processing order is simple: the analysis stage reads the current counts, checks whether the queue is stable, and then applies either the usual wait-time formula or a safer fallback before publishing the metrics.

4.4 Implementation

4.4.1 Queue Event Windows and Rate Estimation

The queue-analysis step uses short time windows to count arrivals and departures. Let N_a and N_d be the number of events observed in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

From these counts, the system estimates the arrival rate and the service rate. To avoid unstable values at the beginning of a run, the implementation waits until enough events have been observed before returning non-zero rates.

4.4.2 Wait-Time Logic and Stability Handling

The queue is considered stable when arrivals stay below service and service is actually observed:

$$\text{queue_stable} = (\lambda < \mu) \wedge (\mu > 0)$$

When that condition is met, the normal wait-time formula is used. Otherwise, the system falls back to a more conservative estimate based on the current occupancy:

- stable queue: $W = \frac{1}{\mu - \lambda}$,
- unstable but service observed: fallback $W = \frac{L}{\mu}$, where L is the number of people in the zone.

When no service events are observed, the implementation returns a zero fallback so that downstream components do not use an unreliable value.

4.4.3 Runtime Contract for Monitoring and Automation

The final metric output includes:

- point estimates (λ, μ, W) ,
- people-in-zone occupancy,
- stability flag for safe downstream interpretation.

These values are then reused by logs, overlays, webhook payloads, and dashboard events.

4.5 Conclusion

Sprint 2 delivered the analytical core of the project. It transforms queue observations into rates, gives a practical wait-time estimate, and keeps a stability signal for the later parts of the system. That makes the next sprints easier to build and validate.

Table of Content

5.1	Introduction	36
5.2	Sprint Backlog	36
5.3	Design	37
5.3.1	Component Diagram	37
5.3.2	Data Flow Diagram	37
5.3.3	Sequence Diagrams	38
5.4	Implementation	38
5.4.1	Camera Source Integration	38
5.4.2	Camera Discovery and Stream Setup	39
5.4.3	API Layer and Data Contract	39
5.4.4	Live Video Delivery	39
5.4.5	Runtime Control and Live Events	40
5.4.6	Persistence and Recovery	41
5.5	Conclusion	41

5.1 Introduction

Sprint 3 expanded the project from a single local pipeline into a platform that can work with several camera sources. At this stage, the backend became responsible for camera onboarding, feed control, and live delivery, so the rest of the application could communicate with it in a more structured way.

5.2 Sprint Backlog

The Sprint 3 backlog focused on the parts needed to connect the system to real camera sources and expose them cleanly to the rest of the application:

- connect to RTSP cameras with authentication, transport control, reconnection, and snapshots,
- discover IP cameras automatically and resolve their stream addresses,
- expose feed management, metadata handling, and diagnostic operations through the API,
- support live delivery through real-time events and a layered video transport strategy.

5.3 Design

5.3.1 Component Diagram

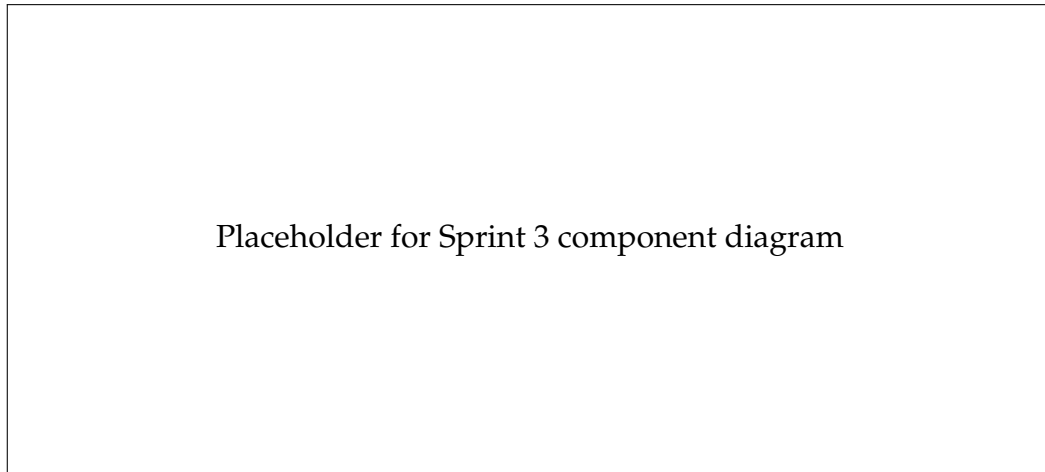


Figure 5.1: Sprint 3 components for camera integration and backend services.

The design keeps camera integration, data handling, runtime control, and live event delivery separate so each part can evolve without forcing changes on the others.

5.3.2 Data Flow Diagram

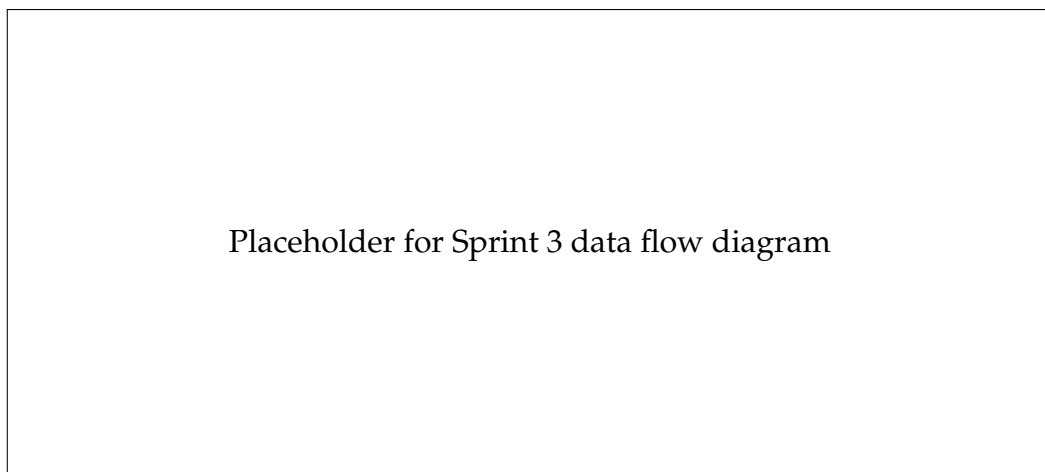


Figure 5.2: Data flow from video feeds to API responses and WebSocket broadcasts.

The main data paths are: first, onboarding and testing flow into the backend; second, feed-control actions update the running state of a camera; and third, runtime events are pushed to the live interface so the dashboard stays in sync.

5.3.3 Sequence Diagrams

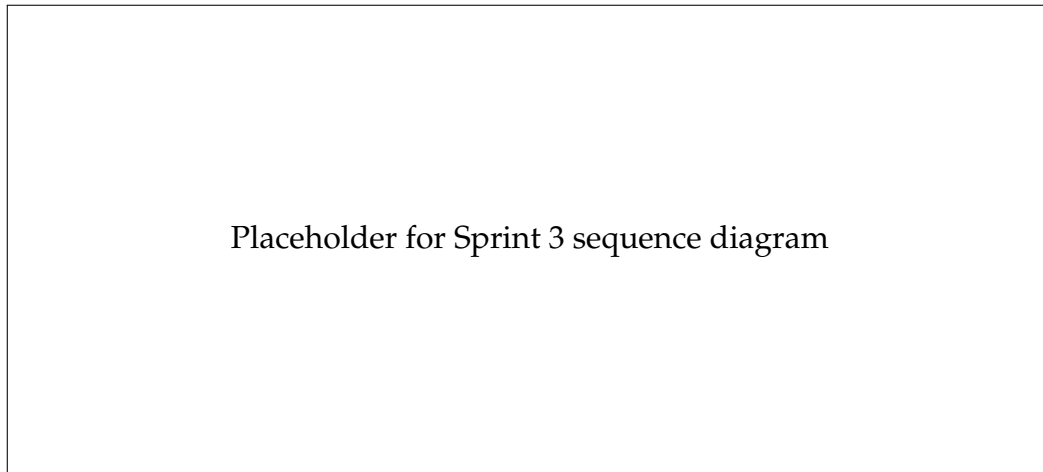


Figure 5.3: Sprint 3 sequence diagram for feed orchestration and live event distribution.

The running sequence is straightforward: a frontend action reaches the backend, the feed state is updated, the worker process changes accordingly, and the new status is sent back to the live interface.

5.4 Implementation

5.4.1 Camera Source Integration

This part of the sprint focuses on making external cameras usable inside the project. It supports:

- handling camera credentials without exposing them in logs,
- choosing between TCP and UDP transport depending on the situation,
- checking the connection and capturing snapshots,

- reconnecting automatically when the stream is interrupted.

The source-handling layer routes local files and webcams through one path and live camera streams through another, which keeps the source logic easier to maintain.

5.4.2 Camera Discovery and Stream Setup

The camera discovery step is used to find available cameras automatically and recover their stream information. This avoids hardcoding stream addresses and makes onboarding easier when new IP cameras are added.

5.4.3 API Layer and Data Contract

The API layer acts as the bridge between the backend services and the rest of the project. It provides structured inputs and outputs so the frontend can interact with the backend in a predictable way. The main groups of operations are:

- feed management, including creation, listing, start, stop, restart, and deletion,
- source onboarding, including RTSP checks, snapshots, and camera discovery,
- metadata management for establishments and cashiers,
- integration and health checks,
- live transport support for real-time viewing.

5.4.4 Live Video Delivery

To reduce viewing latency while still keeping a fallback path, Sprint 3 introduces a transport system that chooses the most suitable stream mode for each feed [5], [7].

For each feed, the backend reports whether live viewing is available through WebRTC or through the fallback video stream. The frontend then uses that information to decide how to display the camera:

- whether the feature is available,
- whether it is ready for immediate use,
- which viewing mode is active,
- why a transport option may be unavailable.

When WebRTC is available, the backend forwards the browser offer to the media server and returns the response needed to start the stream [6]. Before doing that, it checks that the feed exists, is running, and can be used with the selected transport.

Operational status semantics are explicit:

- success when the offer is accepted and the response is returned,
- not found when the feed does not exist,
- conflict when the feed is not running or the transport is not ready,
- invalid input when the stream request is malformed,
- upstream or availability failure when the media service cannot answer.

The configuration stays flexible through environment settings such as the media URL, control URL, timeout, and feature flag. That keeps deployment adaptable without changing the application itself.

5.4.5 Runtime Control and Live Events

Runtime control sits inside the API orchestration layer. It keeps track of feed state, synchronizes persistence, manages the worker lifecycle, and dispatches events to the rest of the system. The live channel publishes updates such as snapshots, feed status changes, metric updates, alert notifications, and system warnings.

For the video surface, the system keeps a shared fallback stream so that viewing does not break when the runtime mode changes.

5.4.6 Persistence and Recovery

The persistence layer stores feed configuration, metadata, video sessions, and archived alerts in SQLite. When the backend starts, it reloads saved feeds and safely resets any stale running sessions so the system begins from a consistent state.

5.5 Conclusion

By the end of Sprint 3, the backend could onboard cameras, manage feed state, and send live updates to the frontend without depending on hardcoded source details. That made the web UI work in Sprint 4 much easier to integrate.

Table of Content

6.1	Introduction	43
6.2	Sprint Backlog	43
6.3	Design	43
6.3.1	UI and UX Design	43
6.3.2	Frontend Architecture	44
6.3.3	Design Rationale	45
6.4	Implementation	45
6.5	Conclusion	45

6.1 Introduction

Sprint 4 defined the visual direction and page structure of the dashboard. The purpose of this sprint was to prepare a clean frontend base before live data and real-time behavior were connected.

6.2 Sprint Backlog

The Sprint 4 backlog focused on turning the design work into a usable frontend structure:

- define a consistent visual style for colors, typography, spacing, and component rules,
- prepare the main page layouts for onboarding, monitoring, editing, analytics, and settings,
- set up the frontend project with reusable interface elements,
- organize the pages and layout so the live integration work could be added later.

6.3 Design

6.3.1 UI and UX Design

The project design specification introduces a “Terminal Minimal” style and a seven-page navigation model. The goal was to give the dashboard a clear and sober visual identity that fits a monitoring application. The design language emphasizes:

- dark operational surfaces optimized for continuous monitoring,
- strong semantic colors for status and alert levels,

- card-based information density with consistent spacing and hierarchy,
- explicit workflow guidance in setup and zone-editing screens.

The designed page set includes the landing page, dashboard, setup wizard, zone editor, analytics, settings, and heatmap views. These screens helped define what the frontend had to support in later stages.

6.3.2 Frontend Architecture

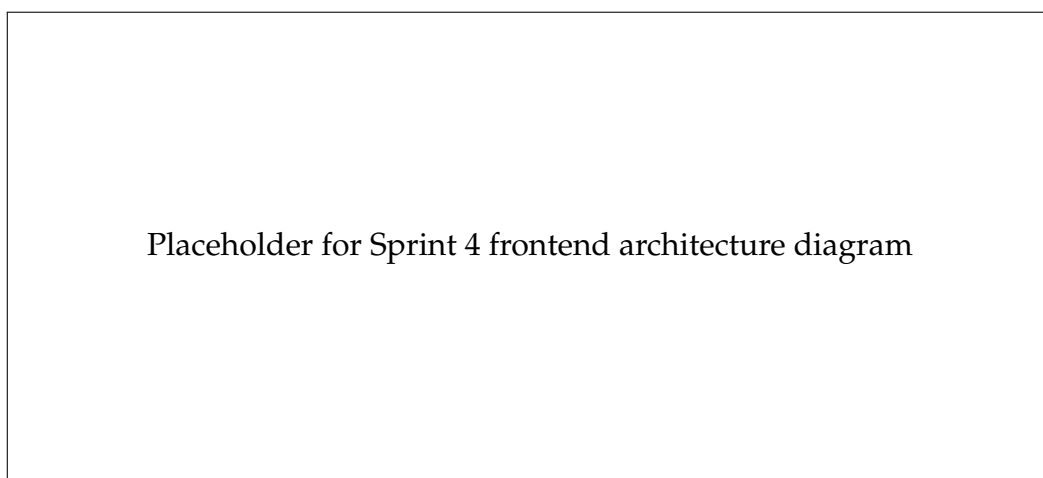


Figure 6.1: Frontend architecture for routing, components, and state boundaries.

The frontend scaffold separates pages, shared interface elements, workflow logic, and the parts that connect the interface to the backend so the dashboard can grow without becoming difficult to maintain:

- page layer: the main screens the user navigates through,
- component layer: reusable interface parts and panels,
- integration layer: the logic that handles data and user actions,
- connection layer: the pieces that connect the frontend to the backend.

6.3.3 Design Rationale

Sprint 4 was intentionally scoped as a foundation sprint rather than a full UI release. The goal was to settle navigation, component boundaries, and interaction patterns before live data and transport logic arrived in Sprint 5. This made the rest of the frontend work easier to organize.

6.4 Implementation

Sprint 4 established the first frontend baseline:

- initial project setup for the frontend workspace,
- a consistent styling approach for the interface,
- a basic page structure with navigation in place,
- starter pages for the dashboard and setup workflow.

The tradeoff in this sprint was deliberate: structure and design consistency came before real-time behavior. That made Sprint 5 easier because the page layout and component boundaries were already in place.

6.5 Conclusion

Sprint 4 left the application with a clear page structure and a stable visual base. That reduced rework when live data arrived in Sprint 5.

Sprint 5: Frontend Integration and Live Monitoring

Table of Content

7.1	Introduction	47
7.2	Sprint Backlog	47
7.3	Design	48
7.3.1	Data Flow Diagram	48
7.3.2	Sequence Diagrams	48
7.4	Implementation	49
7.4.1	API Integration	49
7.4.2	Dashboard State Orchestration	49
7.4.3	Live Update Reliability	49
7.4.4	Live Video Delivery	50
7.4.5	Dashboard UI Integration	50
7.4.6	Integration Testing	51
7.5	Conclusion	51

7.1 Introduction

Sprint 5 connected the dashboard to the live backend services and runtime events. At this stage, the frontend stopped being only a visual shell and became a monitoring interface that could react to feed changes and live metrics.

7.2 Sprint Backlog

The Sprint 5 backlog focused on the pieces needed to make the dashboard react to live data:

- connect the dashboard to backend responses in a reliable way,
- allow the user to manage feeds from the interface,
- receive live updates without refreshing the page,
- choose the most suitable video transport depending on availability,
- keep the dashboard usable when the live connection is interrupted,
- validate the main setup and onboarding flows through integration tests.

7.3 Design

7.3.1 Data Flow Diagram

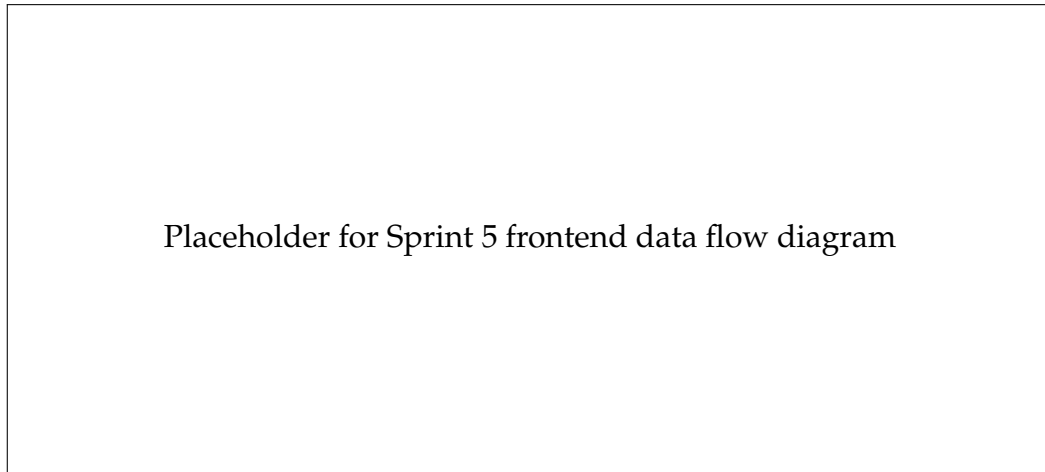


Figure 7.1: Data flow between backend services and frontend monitoring components.

In practice, the backend handles configuration and diagnostic requests, live events carry the real-time updates, and the dashboard groups those signals into KPI cards, activity logs, and attention panels.

7.3.2 Sequence Diagrams

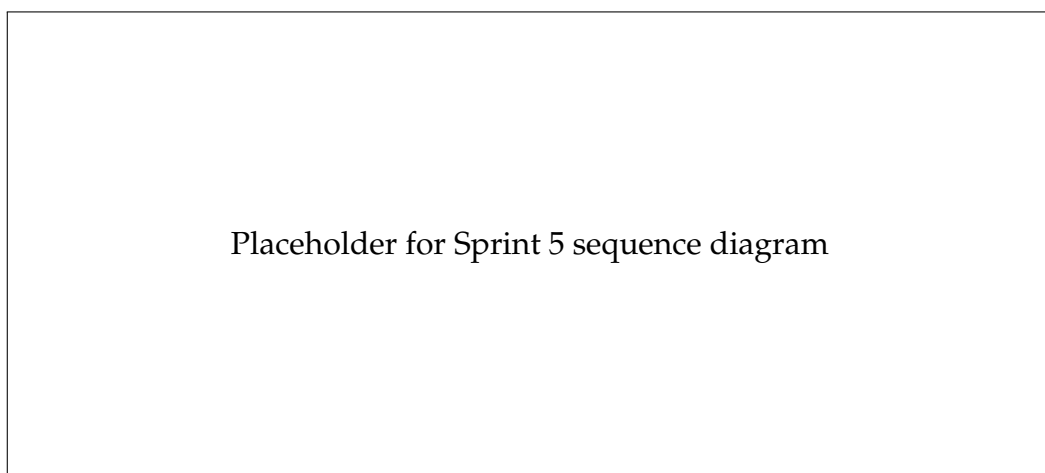


Figure 7.2: Sprint 5 sequence diagram for dashboard updates and alert surfacing.

Three main flows were implemented:

1. **Feed lifecycle:** a dashboard action updates the feed state and the new status is sent back to the interface.
2. **Live metrics:** metric updates are received from the backend and reflected immediately on the dashboard.
3. **Alert surfacing:** alerts reach the interface and appear in the activity and attention areas.

7.4 Implementation

7.4.1 API Integration

The frontend relies on a dedicated API layer that keeps the communication with the backend structured. It defines the main data types used by the dashboard and centralizes request handling so errors can be managed consistently.

7.4.2 Dashboard State Orchestration

A dedicated dashboard state layer coordinates data fetching and updates. From the received feed data, the dashboard derives indicators such as online feeds, total people, average wait time, and attention counts, then exposes them to the widgets.

7.4.3 Live Update Reliability

The live-update layer includes:

- reconnection delays,
- a heartbeat check,
- a guard against stale data,

- fallback polling when live updates fail,
- grouped updates to keep the interface responsive.

When the connection drops, the dashboard falls back to periodic polling instead of freezing, so operators still have a usable view during transient issues.

7.4.4 Live Video Delivery

Live feed cards use a transport decision layer to choose the most suitable viewing mode. For each running feed, the frontend checks whether live viewing is ready. If the higher-quality live stream is available, it is used first; otherwise the dashboard falls back to the simpler video stream.

The live-streaming flow is handled through the browser's live video connection support [5]:

- create a connection for receiving video,
- gather the network information needed for signaling,
- send the browser offer to the backend,
- apply the backend response and switch the feed card to live state.

Reconnection uses increasing delays between attempts. When the problem is not recoverable, the system stops retrying and switches to the fallback stream so the dashboard remains usable.

When server-side annotations are active, the dashboard hides local overlays to avoid drawing the same boxes twice on the same video.

7.4.5 Dashboard UI Integration

The dashboard page brings together live feed controls, setup dialogs, source onboarding, KPI visualizations, and charts. In this sprint, the emphasis was on making those

elements work together as one operational interface.

7.4.6 Integration Testing

Frontend integration behavior is validated through scenario-based tests that cover dashboard setup, live event handling, transport fallback, feed routing, and error behavior.

7.5 Conclusion

By the end of Sprint 5, the dashboard could react to live state, switch transport modes, and stay usable when the live connection failed. Sprint 6 then built on that working interface for alert automation.

Table of Content

8.1	Introduction	53
8.2	Sprint Backlog	53
8.3	Design	54
8.3.1	Component Diagram	54
8.3.2	Data Flow Diagram	54
8.4	Implementation	55
8.4.1	Alert Payload Structure	55
8.4.2	Delivery Reliability	55
8.4.3	Alert Dispatch in the Runtime Loop	55
8.4.4	n8n Workflow Routing and Telegram Delivery	56
8.4.5	Workflow Validation	56
8.5	Conclusion	56

8.1 Introduction

This sprint connected the queue metrics produced by the backend with n8n, so alerts could be forwarded in a controlled way. The main concern was not only to transmit the payload, but also to keep the process reliable, traceable, and easy to repeat during testing.

8.2 Sprint Backlog

Sprint 6 focused on the automation tasks needed to move alerts from the backend into n8n:

- define a stable webhook payload and validate it before dispatch,
- add retry-based webhook delivery with logging for failures and successes,
- connect runtime threshold detection to alert triggering,
- configure n8n routing for authorization, cooldown, formatting, and delivery,
- prepare test scripts for repeated workflow validation.

8.3 Design

8.3.1 Component Diagram

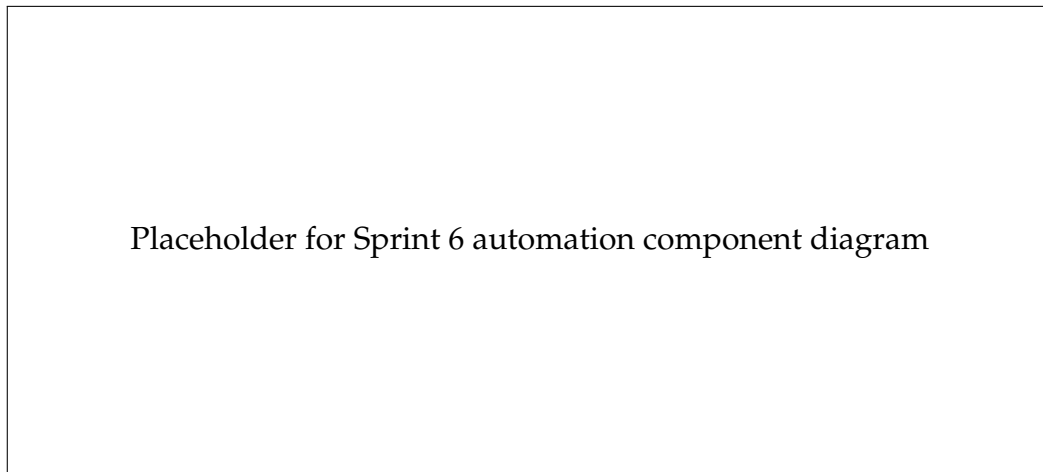


Figure 8.1: Components involved in backend-to-n8n-to-Telegram automation flow.

The automation chain is made up of the runtime metrics collector, the webhook serializer, the HTTP delivery client, the n8n workflow, and the final notification channel. When an alert needs to be preserved for later review, it is sent back to the backend through a dedicated archive endpoint.

8.3.2 Data Flow Diagram

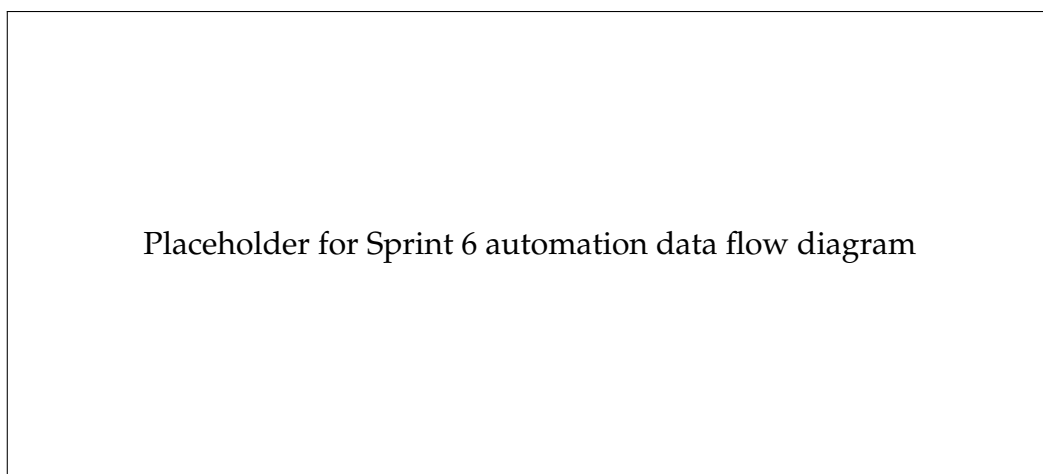


Figure 8.2: Alert payload flow from backend metrics to external notifications.

The process begins when the runtime loop sends metrics at regular intervals. n8n then checks whether an alert is present and whether the cooldown allows delivery. If the alert can be sent, Telegram receives the notification; otherwise the workflow only acknowledges the request. Archived alert payloads are stored through `/api/alerts/archive`.

8.4 Implementation

8.4.1 Alert Payload Structure

The webhook schema is centered on `QueuePayload`, which keeps the information needed by n8n in one structure. It carries:

- source metadata (timestamp, frame, source, feed ID),
- queue metrics (λ , μ , wait time, stability),
- alert flags and severity fields.

Before anything is sent, the payload is validated so malformed records do not leave the backend.

8.4.2 Delivery Reliability

To reduce the risk of missed notifications, the delivery client uses a timeout, a retry policy, an optional shared-secret header, and logging for both successes and failures. Historical payloads remain in SQLite, which gives the system one place to recover past events if needed.

8.4.3 Alert Dispatch in the Runtime Loop

Inside the main loop, alert sending is moved to `AsyncWebhookDispatcher` so the perception pipeline keeps running without waiting on the network. This makes it possible

to send periodic metric updates and immediate alerts, while a deduplication window limits repeated notifications for the same situation.

8.4.4 n8n Workflow Routing and Telegram Delivery

The n8n alert workflow follows a simple path:

- webhook reception and secret validation,
- alert detection and cooldown gating,
- alert history archival via backend API,
- message formatting and Telegram send node,
- explicit acknowledge/reject branches.

Keeping the no-alert, alert-suppressed, and alert-delivered branches separate makes the behavior easier to audit later.

8.4.5 Workflow Validation

Workflow validation was supported by PowerShell scripts for single-alert checks, automated execution polling, and multi-feed scenarios. These scripts made it possible to replay webhook submissions and confirm that each branch behaved as expected during sprint verification.

8.5 Conclusion

Sprint 6 turned the analytics output into notifications that could reach the operations side, while still preserving a clear history inside the backend.

Sprint 7: Evaluation and Optimization

Table of Content

9.1	Introduction	58
9.2	Sprint Backlog	58
9.3	Design	58
9.3.1	Verification Plan	58
9.3.2	Performance Targets	59
9.4	Implementation	59
9.4.1	Verification Coverage	59
9.4.2	Runtime Tuning	59
9.4.3	Fault-Tolerance Improvements	60
9.4.4	Overall Outcome	60
9.5	Conclusion	60

9.1 Introduction

This sprint served as the final validation step for the full stack. It checked whether perception, the API layer, the frontend, and the alerting path still worked together when the system was tested under retries, changing feeds, and higher throughput.

9.2 Sprint Backlog

Sprint 7 focused on the last round of verification and improvement work:

- bring backend and frontend automated tests together into one verification pass,
- confirm feed lifecycle behavior, stream continuity, and API contracts,
- improve runtime throughput and responsiveness,
- strengthen failure handling for network drops, stale data, and worker exits,
- record the remaining limitations before final delivery planning.

9.3 Design

9.3.1 Verification Plan

Validation followed a layered strategy that covered the system from the backend to the dashboard:

- **Backend unit/integration tests:** queue analytics logic, API endpoints, runtime streaming behavior, and webhook payload handling.
- **Frontend integration tests:** dashboard setup flow, websocket event handling, media fallback behavior, and API error handling.

- **Scenario tests:** n8n send/acknowledge flows, unauthorized access paths, and multi-feed alert scenarios.

9.3.2 Performance Targets

The optimization work was guided by the main operational requirements of the project:

- processing responsiveness, especially loop cadence and processed-frame throughput,
- stream continuity when a feed changes state,
- webhook impact on the main-loop latency,
- dashboard freshness when websocket quality degrades.

9.4 Implementation

9.4.1 Verification Coverage

The test suite covers API transport behavior, runtime stream handling, webhook delivery, dashboard integration, live-state synchronization, API error cases, and transport fallback behavior. Taken together, these tests check the API envelope, the event contract, stream handoff, and the setup workflow.

9.4.2 Runtime Tuning

The main tuning actions applied across the runtime loop and configuration layers included:

- configurable detector image size and frame stride,
- explicit control over the inference device and its diagnostics,
- asynchronous webhook dispatch so the processing loop stayed responsive,

- real-time playback pacing for file-based sources.

9.4.3 Fault-Tolerance Improvements

Runtime resilience was improved at several levels:

- feed worker supervision and exit recovery in `FeedRegistry`,
- capture-to-worker frame handoff continuity in `FeedFrameStreamManager`,
- websocket reconnect and fallback behavior on the frontend side.

9.4.4 Overall Outcome

By the end of the sprint, the system had reached a stable integration point where perception, analytics, the API runtime, the dashboard, and n8n automation operated as one pipeline. What remains is mainly benchmarking and deployment hardening, not missing core functionality.

9.5 Conclusion

Sprint 7 closed the project with structured testing and targeted performance improvements. At this stage, the platform is strong enough for demonstration and academic evaluation, while the remaining work is mainly about deployment hardening and longer-term benchmarking.

Table of Content

10.1 Summary	62
10.2 Objective Review	62
10.3 Future Work	63

10.1 Summary

This end-of-studies project produced a complete queue wait-time estimation platform that brings together computer vision, queue analytics, web supervision, and automation workflows.

In practical terms, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- stability-aware outputs with explicit queue-state semantics,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live web-socket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

The resulting architecture shows that a full monitoring-to-action loop can be built from modular components without losing maintainability or testability.

10.2 Objective Review

The initial objectives were achieved at the project scope level:

1. **Real-time detection and tracking:** achieved through dedicated detector and tracker modules.
2. **Queue-zone counting:** achieved with polygon-zone management and in-zone identity filtering.
3. **Queue metric estimation:** achieved for arrival rate, service rate, and wait time.

4. **Queue stability handling:** achieved through stable and unstable classification with fallback wait-time logic.
5. **Web monitoring:** achieved through a multi-feed dashboard with live updates.
6. **Automation integration:** achieved through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but the depth of long-horizon evaluation under broader operational scenarios.

10.3 Future Work

Possible extensions include:

- multi-zone and multi-server queue modeling for parallel cashier configurations,
- richer alert strategies with cooldown tuning, escalation policies, and acknowledgment loops,
- edge deployment optimization through ONNX/TensorRT profiling and resource-aware scheduling,
- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation*, <https://docs.ultralytics.com/>, 2026.
- [3] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [4] OpenCV, *Opencv documentation*, <https://docs.opencv.org/>, 2026.
- [5] W3C Web Real-Time Communications Working Group, *WebRTC 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-04-07, 2025.
- [6] MediaMTX, *Mediamtx documentation*, <https://mediamtx.org/docs/>, Accessed: 2026-04-07, 2026.
- [7] Internet Engineering Task Force, *Rfc 8830: WebRTC media transport and use of rtp*, <https://www.rfc-editor.org/rfc/rfc8830>, Accessed: 2026-04-07, 2021.



A.1 Useful Commands

- Run backend API: `uvicorn backend.app:app -reload`
- Run backend tests: `pytest tests -q`
- Run frontend dev server: `cd frontend && npm run dev`
- Run frontend tests: `cd frontend && npm run test`

A.2 Example Webhook Payload

```
{  
  "timestamp": "2026-03-05T14:32:10.123Z",  
  "camera_id": "cam_01",  
  "zone_id": "checkout_lane_3",  
  "metrics": {  
    "people_in_zone": 7,  
    "arrival_rate": 0.15,  
    "service_rate": 0.16,  
    "wait_time_seconds": 15.0,  
    "queue_stable": true  
  },  
  "alerts": [  
    {
```

```
    "type": "WAIT_TIME_WARNING",
    "severity": "warning",
    "message": "Wait time exceeded 60s threshold",
    "value": 72.3,
    "threshold": 60
  }
],
"raw_detection_count": 7,
"fps": 24.5
}
```

A.3 Items to Add Before Final Submission

- final versions of all architecture and sequence diagrams,
- finalized evaluation tables and plots for Sprint 7,
- appendix references for API and test artifacts,
- proofreading pass for terminology and citation consistency.